

Session 7

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva

1



RASPBERRY PI

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

2

RASPBERRY PI

- Microcontroller based boards
 - Raspberry Pi Pico 2
 - 2 x Arm Cortex-M33@150MHz and 2 x RISC-V (Hazard3)
 - Only two cores can be used at each instant. Usually chosen at boot time.
 - RISC-V: open-source hardware, development/educational purposes (but worse FPU performance)
 - Up to 520 KB SRAM, 2 x UART, 2 x SPI, 2x I2C, 16 PWM channels, USB 1.1. controller, ADC (some models)
 - Raspberry Pi Pico – dual core Arm Cortex-M0+ processor with 264kB internal RAM
- Solutions based on Application Microprocessor (with MMU, SoC)
 - Raspberry Pi Compute Module 5
 - Broadcom BCM2712 quad-core Arm Cortex-A76 @ 2.4GHz
 - Raspberry Pi 5 (BCM2712 @ 2.5 GHZ) and 4
 - Raspberry Pi 3, 2, 1 are discontinued
 - Raspberry Pi Zero 2 (4 x Arm Cortex-A53@1GHZ),
 - Raspberry Pi Zero (1 GHz single-core CPU)



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

3

3

COMPUTE MODULE DEVELOPMENT KIT



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

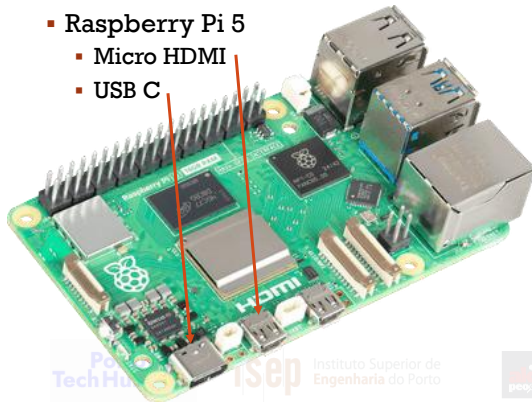
4

4

OTHER FORM FACTORS

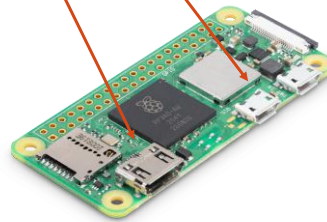
- Raspberry Pi 5

- Micro HDMI
- USB C



- Raspberry Pi Zero 2 W

- Micro USB
- Mini HDMI



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

5

5

EVOLUTION OF RASPBERRY PI SOC

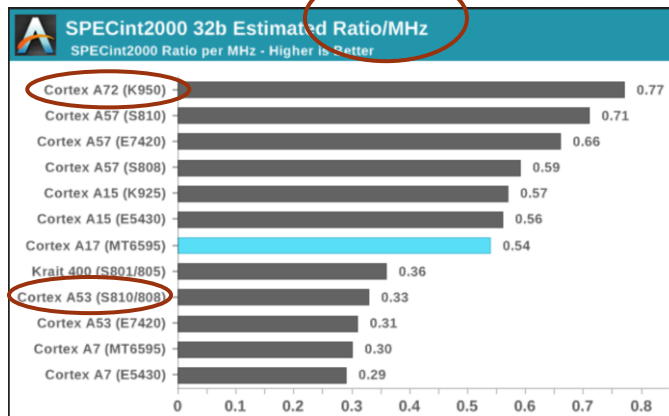
- Raspberry Pi 3 (2016): BCM2837– 4 x ARM Cortex-A53 @ 1.4GHz
 - Cortex-A53 (ARMv8-A ISA) - Power efficiency (the “LITTLE” in Arm big.LITTLE)
 - 2-way superscalar processor with in-order execution
- Raspberry Pi 4 (2019): BCM2711 - 4 x ARM Cortex-A72 @ 1.5 GHZ
 - Cortex-A72 (ARMv8-A ISA) – Designed for performance (“big”)
 - 3-way superscalar processor with out-of-order execution
 - About 50% performance improvement relatively to the Cortex-A53 on the BCM2837
- Raspberry Pi 5 (2023): BCM2712 - 4 x ARM Cortex-A76 @ 2.4 GHZ
 - Cortex-A76 (ARMv8.4 ISA)
 - 4-way superscalar processor with out-of-order execution
 - Change from 28 nm (used on BCM2711) to 16 nm process and other microarchitecture changes:
 - Higher performance per watt than Cortex-A72 on BCM2711 (almost double)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

6

6

PERFORMANCE COMPARISON



- Integer performance
 - Standard Performance Evaluation Corporation (SPEC)
 - Results normalized with respect to CPU frequency

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

7

7

DEVELOPMENT FOR THE RASPBERRY PI PLATFORM (EXCEPT FOR PICO)

- Approaches
 - Development on the Raspberry Pi platform, with native C compiler (code compiled on the Raspberry Pi)
 - Slower than most personal computers
 - Necessary to install development tools (does not make much sense for a target system)
 - Use a cross-compiling toolchain
 - Pre-compiled toolchains: Linaro, etc.
 - Attention to library versions/dependencies
 - Toolchain libraries (.so files) must be compatible with the libraries used in the Raspberry Pi Linux distribution
 - Generate custom distribution and toolchain, using tools such as Buildroot or Yocto.
 - Select only required application and operating system services
 - Compile from source and optimize for target CPU

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

8

8

DEVELOPMENT FOR THE RASPBERRY PI PLATFORM

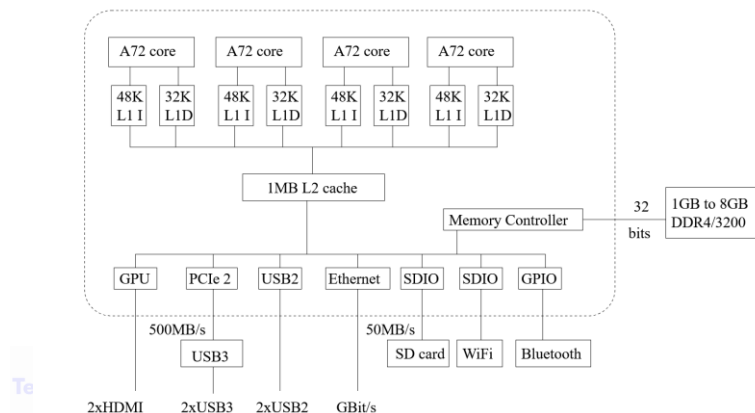
- You should have a Linux system on your personal computer.
 - You may use a virtual machine.
 - WSL (Windows 10 and 11), Hyper-V (Windows 10 and 11 Pro), VirtualBox, etc.
 - <https://learn.microsoft.com/en-us/windows/wsl/install>
 - <https://www.virtualbox.org/wiki/Downloads>
 - Macs with ARM CPU (most models since 2021): choose a distribution with support for ARM aarch64.
 - For instance, <https://www.fedoraproject.org/workstation/download/>
- Please bring a USB pen drive (preferably of 4 GB or more) to the PL classes

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

9

9

RASPBERRY PI 4 SOC: BCM2711



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

10

10

LEGAL NOTICE

- *“HIGH RISK ACTIVITIES. Raspberry Pi products are not designed, manufactured or intended for use in hazardous environments requiring fail safe performance, such as in the operation of nuclear facilities, aircraft navigation or communication systems, air traffic control, weapons systems or safety-critical applications (including life support systems and other medical devices), in which the failure of the products could lead directly to death, personal injury or severe physical or environmental damage (“High Risk Activities”). RPL specifically disclaims any express or implied warranty of fitness for High Risk Activities and accepts no liability for use or inclusions of Raspberry Pi products in High Risk Activities”*
- BCM2711 ARM Peripherals, <https://pip.raspberrypi.com/documents/RP-008248-DS-bcm2711-peripherals.pdf>, accessed in 2026/03/24



Instituto Superior de Engenharia do Porto



Santander

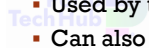
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

11

11

BCM2711

- 28 GPIO
 - Up to 6x UART
 - Up to 6x I2C
 - Up to 5x SPI
 - Up to 2x PWM channels
 - 1x SDIO interface
 - 1x PCM (audio, serial input/output)
 - 1x DPI (Parallel RGB Display)
 - Up to 3x General Purpose GPIO Clocks (GPCLK) outputs
- DMA support
 - Used by the SD Card controller.
 - Can also be used by some UARTS and SPI



Instituto Superior de Engenharia do Porto



Santander

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

12

12

ADDITIONAL “SPECIFICATIONS”

- No ADC...
- No high frequency external input for counters
 - It is possible to count impulses by enabling interrupts, but it is limited to a few kHz
- Possible solution: use an external device (a microcontroller, for instance, or a HAT) connected through SPI or I2C



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

13

13

RASPBERRY PI RTC

- No battery!
 - `ntpdate -u -s time.nist.gov`
 - `-u`: Use an unprivileged port for outgoing packets, critical when running behind a firewall that blocks privileged ports.
 - `-s`: Divert logging output from standard output to the system syslog facility, ideal for cron jobs.
 - If no Internet connection:
 - `timedatectl set-time 'YYYY-MM-DD hh:mm:ss'`



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

14

14

Diagram of the Raspberry Pi Header (J8) showing pin connections for GPIOs, 3V3, and 5V. The header has 40 pins. Pins 1, 2, 3, 4 are 3V3. Pins 5, 6, 7, 8 are 5V. Pins 9-20 are GPIOs. Pins 21-26 are ID_SD, ID_SC, and other functions. Pins 27-39 are GPIOs. Pins 40 is 5V.

RASPBERRY PI HEADER

- GPIO0 to GPIO27 accessible
- GPIO 0 and 1 are always used in alternate function ID_SD and ID_SC (I2C)
 - Reserved for connecting to "Hardware Attached on Top" (HAT) boards

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

15

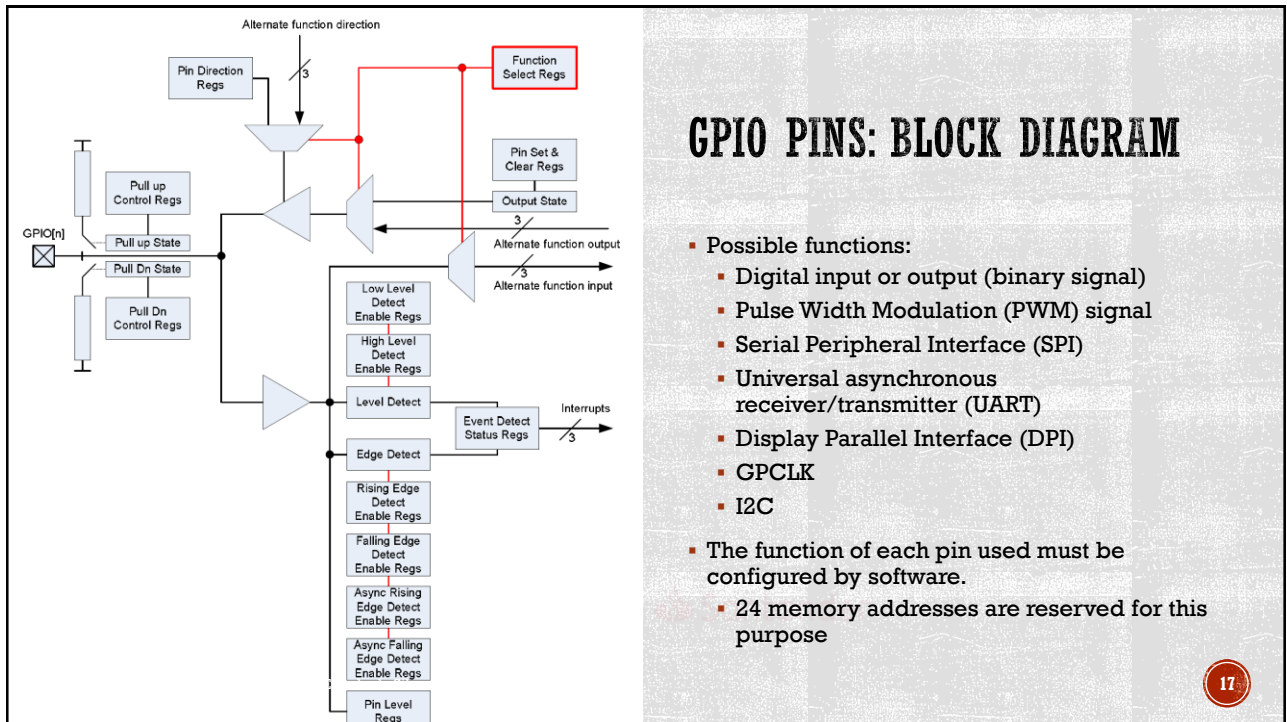
Diagram of the Raspberry Pi Header (J8) showing pin connections for alternate functions. The header has 40 pins. Pins 1, 2, 3, 4 are 3.3V. Pins 5, 6, 7, 8 are 5V. Pins 9-20 are GPIOs. Pins 21-26 are ID_SD, ID_SC, and other functions. Pins 27-39 are GPIOs. Pins 40 is 5V.

ALTERNATE FUNCTIONS (RPI ORIGINAL HEADER)

- Some of the "alternate" functions (DPI, SPI) require several GPIO to be jointly configured for that function

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

16



17

MAP OF CONFIGURATION REGISTERS

Bit	Endereço relativo	
	0x200000	0x200004
0	GPIO 0	GPIO10
1		
2		
3		
4	GPIO 1	
5		
6		
7	GPIO 2	
8		
9		
10	GPIO 3	
11		
12		
13	GPIO 4	
14		
15		
16	GPIO 5	
17		
18		
19	GPIO 6	
20		
21		
22	GPIO 7	
23		
24		
25	GPIO 8	
26		
27		
28	GPIO 9	
29		
30		
31		

Every 4 bytes are used to configure 10 GPIO pin

3 bits per pin:

- 000 for digital input
- 001 for digital output
- 100 for alternate function 0
- 101 for alternate function 1
- 110 for alternate function 2
- 111 for alternate function 3
- 011 for alternate function 4
- 010 for alternate function 5

Not all pins have 6 alternate functions

Function "N" may vary from pin to pin

18

DIGITAL OUTPUT MODE

- Even more "compact": 8 bytes, where each bit corresponds to one of the lines
 - Bit 0 of the base address corresponds to GPIO 0
 - Bit 7 of the last address corresponds to GPIO 31
 - Writing 1 to the addresses `GPIO_BASE+[0x001C to 0x0023]` puts line 1
 - Writing 1 to the addresses `GPIO_BASE+[0x0028 to 0x002F]` puts line 0
- Why not use an address for each pin?
 - To save addresses from main memory address space (artifact from 32-bit architectures)



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

19

19

USING GPIO ON LINUX

- Linux already has support for most features
 - Most GPIO functions are accessed through files
 - Libraries to wrap access to those files do exist (WiringPi, for instance)
 - <https://github.com/wiringpi/wiringpi>
- But someone had to wrote the corresponding device drivers...



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

20

20

MAPPING REGISTERS TO C STRUCT

```
typedef struct {
    uint32_t GPFSEL[6];
    uint32_t Reserved1;
    uint32_t GPSET[2];
    uint32_t Reserved2;
    uint32_t GPCLR[2];
    uint32_t Reserved3;
    uint32_t GPLEV[2];
    ...
} GpioRegs_t;
```

Offset	Name	Description
0x00	GPFSEL0	GPIO Function Select 0
0x04	GPFSEL1	GPIO Function Select 1
0x08	GPFSEL2	GPIO Function Select 2
0x0c	GPFSEL3	GPIO Function Select 3
0x10	GPFSEL4	GPIO Function Select 4
0x14	GPFSEL5	GPIO Function Select 5
0x1c	GPSET0	GPIO Pin Output Set 0
0x20	GPSET1	GPIO Pin Output Set 1
0x28	GPCLR0	GPIO Pin Output Clear 0
0x2c	GPCLR1	GPIO Pin Output Clear 1
0x34	GPLEV0	GPIO Pin Level 0
0x38	GPLEV1	GPIO Pin Level 1

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

21

21

ACCESING GPIO REGISTERS FROM USER SPACE

```
#define RPI4_PERIPH_BASE 0xFE000000
#define RPI4_GPIO_BASE (RPI4_PERIPH_BASE + 0x200000)

mem_fd = open("/dev/mem", O_RDWR|O_SYNC); //O_SYNC ensures written data is flushed

//gpio_regs will point to desired physical address (GPIO_BASE, in this example)
GpioRegs_t *gpio_regs = mmap(
    NULL,
    ADDRESS_RANGE, //Number of bytes to be addressed (e.g., number of 8 bit registers)
    PROT_READ|PROT_WRITE, MAP_SHARED,
    mem_fd,          // File descriptor to physical memory virtual file '/dev/mem'
    RPI4_GPIO_BASE  // Address in physical map that we want to map and access
);
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

22

22

/DEV/MEM

- Shortcomings
 - User processes cannot use interrupts
 - Must be careful to check if loaded device drivers are not accessing the same registers
 - User processes may be preempted (interrupted by the kernel) in a critical section



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

23

23

Session 8

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva

24

GPIO DIGITAL INPUT/OUTPUT FROM THE LINUX SHELL

- Set GPIO17 as output: `pinctrl set 17 op`
- Set GPIO17 to 1: `pinctrl set 17 dh ("drive high")`
- Set GPIO17 to 0: `pinctrl set 17 dl ("drive low")`
- Set GPIO17 as input: `pinctrl set 17 ip ("drive low")`



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

25

25

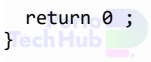
WIRINGPI EXAMPLE: LED BLINKING

```
#include <wiringPi.h>
#include <stdlib.h>
#include <stdio.h>

#define ledPin 17

int main() {
    wiringPiSetupGpio();
    pinMode(ledPin, OUTPUT);
    while (1) {
        delay(100);
        digitalWrite(ledPin, 0);
        delay(100);
        digitalWrite(ledPin, 1);
    }

    return 0;
}
```



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

26

26

WIRINGPI EXAMPLE: LED BLINKING

```
#include <wiringPi.h>

int main(void) {
    wiringPiSetupGpio();

    pinMode(17, INPUT); // set pin 17 to input (INPUT, OUTPUT, PWM_OUTPUT, GPIO_CLOCK)

    // pull up/down mode (PUD_OFF, PUD_UP, PUD_DOWN) => down
    pullUpDnControl(17, PUD_DOWN);

    int value = digitalRead(17); // get state of pin 17

    if (HIGH == value)
    {
        // your code
    }
}
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

27

27

WIRINGPI APPROACH

- wiringpi.c: wiringPiSetup(). Code fragment:


```
const char* gpiomem_global = "/dev/mem";
( ... )
fd = open (gpiomemGlobal, O_RDWR | O_SYNC | O_CLOEXEC)
( ... )
GPIO_BASE = piGpioBase + 0x00200000 ;
( ... )
gpio = (uint32_t *) mmap(0, BLOCK_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
GPIO_BASE) ;
```
- O_CLOEXEC – file is closed when one the exec functions is used

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

28

28

WIRING PI

- git clone <https://github.com/WiringPi/WiringPi>
- cd WiringPi
- ./build debian
- mv debian-template/wiringpi-3.x.deb .
- sudo apt install ./wiringpi-3.x.deb



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

29

29

RASPBERRY GPIO ALTERNATE FUNCTIONS

- Can be checked using the **pinctrl** command line utility



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

30

30

pinctrl -c bcm2711

0, GPIO0, SDA0, SA5, PCLK, SPI3_CE0_N, TXD2, SDA6
 1, GPIO1, SCL0, SA4, DE, SPI3_MISO, RXD2, SCL6
 2, GPIO2, SDA1, SA3, LCD_VSYNC, SPI3_MOSI, CTS2, SDA3
 3, GPIO3, SCL1, SA2, LCD_HSYNC, SPI3_SCLK, RTS2, SCL3
 4, GPIO4, GPCLK0, SA1, DPI_D0, SPI4_CE0_N, TXD3, SDA3
 5, GPIO5, GPCLK1, SA0, DPI_D1, SPI4_MISO, RXD3, SCL3
 6, GPIO6, GPCLK2, SOE_N_SE, DPI_D2, SPI4_MOSI, CTS3, SDA4
 7, GPIO7, SPI0_CE1_N, SWE_N_SRW_N, DPI_D3, SPI4_SCLK, RTS3, SCL4
 8, GPIO8, SPI0_CE0_N, SD0, DPI_D4, I2CSL_CE_N, TXD4, SDA4
 9, GPIO9, SPI0_MISO, SD1, DPI_D5, I2CSL_SDI_MISO, RXD4, SCL4
 10, GPIO10, SPI0_MOSI, SD2, DPI_D6, I2CSL_SDA_MOSI, CTS4, SDA5
 11, GPIO11, SPI0_SCLK, SD3, DPI_D7, I2CSL_SCL_SCLK, RTS4, SCL5
 12, GPIO12, PWM0_0, SD4, DPI_D8, SPI5_CE0_N, TXD5, SDA5
 13, GPIO13, PWM0_1, SD5, DPI_D9, SPI5_MISO, RXD5, SCL5

31

14, GPIO14, TXD0, SD6, DPI_D10, SPI5_MOSI, CTS5, TXD1
 15, GPIO15, RXD0, SD7, DPI_D11, SPI5_SCLK, RTS5, RXD1
 16, GPIO16, -, SD8, DPI_D12, CTS0, SPI1_CE2_N, CTS1
 17, GPIO17, -, SD9, DPI_D13, RTS0, SPI1_CE1_N, RTS1
 18, GPIO18, PCM_CLK, SD10, DPI_D14, SPI6_CE0_N, SPI1_CE0_N, PWM0_0
 19, GPIO19, PCM_FS, SD11, DPI_D15, SPI6_MISO, SPI1_MISO, PWM0_1
 20, GPIO20, PCM_DIN, SD12, DPI_D16, SPI6_MOSI, SPI1_MOSI, GPCLK0
 21, GPIO21, PCM_DOUT, SD13, DPI_D17, SPI6_SCLK, SPI1_SCLK, GPCLK1
 22, GPIO22, SD0_CLK, SD14, DPI_D18, SD1_CLK, ARM_TRST, SDA6
 23, GPIO23, SD0_CMD, SD15, DPI_D19, SD1_CMD, ARM_RTCK, SCL6
 24, GPIO24, SD0_DAT0, SD16, DPI_D20, SD1_DAT0, ARM_TDO, SPI3_CE1_N
 25, GPIO25, SD0_DAT1, SD17, DPI_D21, SD1_DAT1, ARM_TCK, SPI4_CE1_N
 26, GPIO26, SD0_DAT2, -, DPI_D22, SD1_DAT2, ARM_TDI, SPI5_CE1_N
 27, GPIO27, SD0_DAT3, -, DPI_D23, SD1_DAT3, ARM_TMS, SPI6_CE1_N

32

EXAMPLE: CHOOSING I2C AS ALTERNATE FUNCTION FOR GPIO 4 AND 5

- After each boot (can be automated)
 - `sudo modprobe i2c-dev`
 - `sudo modprobe i2c-bcm2835`
- Assuming you want to bind I2C3 to GPIO4 and GPIO5
 - `sudo dtoverlay i2c-gpio bus=3 i2c_gpio_sda=4 i2c_gpio_scl=5`
 - Can be automatically loaded by adding the respective entry on config.txt, on Raspberry Pi boot partition
- `sudo dtoverlay -l`
 - 0: `i2c-gpio bus=3 i2c_gpio_sda=4 i2c_gpio_scl=5`
- To remove overlay:
 - `sudo dtoverlay -r 0`

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

33

33

dtoverlay - load a device-tree overlay

SYNOPSIS

```
dtoverlay [-d dir] [-p string] [-v] [-D] [-r] [-R] [-l] [-a] [overlay
[param=val...]]
```

```
dtoverlay -h [overlay [param]]
```

DESCRIPTION

dtoverlay is a command line utility for manipulating the system's runtime device-tree by adding or removing overlays. It can also customize the base device-tree or overlays by setting parameters within them. See [DTREE] for more information.

By default, without the -r or -R options, the application adds the specified overlay. Manipulation of active device-tree overlays usually requires root privileges.

Porto TechHub

isep Instituto Superior de Engenharia do Porto

pgsce

Santander

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

34

34

A SIMPLE I2C TARGET

- Note that Raspberry Pi GPIO pins do not tolerate voltages above 3.3 V
 - Do not connect directly to Arduino UNO or any other device using 5 V for high level.

```
#include <Wire.h>

uint8_t Byte = 1;

void I2C_RxHandler(int numBytes) {
    while(Wire.available()) {
        Byte = Wire.read();
    }
}

void I2C_TxHandler(void) {
    Wire.write(Byte);
}

void setup() {
    Wire.begin(0x55); // address=0x55
    Wire.onReceive(I2C_RxHandler);
    Wire.onRequest(I2C_TxHandler);
}

void loop() {}
```



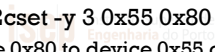
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

35

35

I2C TOOLS

- I2c-tools packages provide several commands to interact with I2C bus:
 - `sudo apt install i2c-tools`
- `i2cdetect 3`
 - detect devices on bus 3
- `i2cget`
 - Example: `i2cget -y 3 0x55`
 - Read data from device 0x55 on bus 3
- `i2cset`
 - Example: `i2cset -y 3 0x55 0x80`
 - Send value 0x80 to device 0x55 on bus 3



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

36

36

RASPBERRY PI DEFAULT UART (GPIO14:15)

- Associated to device file /dev/ttyS0
- systemctl stop [serial-getty@ttyS0.service](#)
 - To stop typical getty process on /dev/ttyS0
- lsof /dev/ttyS0
 - To confirm there is no process accessing /dev/ttyS0
- stty -F /dev/ttyS0 sane
 - Set serial communication parameters to the defaults
- stty -F /dev/ttyS0 9600 -echo
 - Make sure baud rate is 9600 and disable echo
- cat /dev/ttyS0
- echo test > /dev/ttyS0
 - Send some data from the command line

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

37

37

LINUX SERIAL PORT CONFIGURATION IN C

- Use functions from termios.h
 - More than 50 adjustable parameters
 - Get/set Linux serial interface configurations: tcgetattr(3), tcsetattr(3)
 - Baud rate helpers: cfgetispeed(), cfgetospeed(), cfsetispeed(), cfsetospeed()
 - options.c_lflag &= ~ICANON; //disable line-oriented discipline (do not wait for \r or \n)
 - options.c_cc[VTIME]=0; //disable receive timeout (for blocking mode)
 - options.c_cc[VMIN]=1; //minimum number of characters to receive before read returns
- man termios
- <https://pubs.opengroup.org/onlinepubs/000095399/basedefs/termios.h.html>
- https://en.wikibooks.org/wiki/Serial_Programming/termios

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

38

38